ARTIFICIAL INTELLIGENCE
SEARCH AND OPTIMIZATION TECHNIQUES

ASSIGNMENT

Submitted by
POOJA SIVA
Register Number: 192419057

Department: _____BME_____
Institution: ____SIMATS_____
Academic Year: __2024_____

CONTENTS

- 1. AI-Powered Drone Delivery in Flood-Affected Regions
- 2. Smart City Traffic Signal Optimization
- 3. Autonomous Rover Exploring Mars
- 4. University Examination Timetable Using CSP
- 5. Strategic Game AI Using Minimax and Alpha-Beta Pruning
- Overall Conclusion

1. AI-Powered Drone Delivery in Flood-Affected Regions

Introduction

An AI-powered drone is used to deliver emergency medicines to people in flood-affected areas. Roads and paths may become blocked, while weather conditions can change travel time, energy use, and risk. Greedy Best-First Search and A* Search can help the drone choose a route to its destination.

1.1 Greedy Best-First Search

Greedy Best-First Search is an informed search algorithm that selects the next node using a heuristic estimate of how close it is to the goal.

Heuristic function: h(n) = estimated distance from node n to the destination.

Working:

1. The drone identifies its current location and destination.
2. It estimates the straight-line distance from each available location to the destination.
3. It selects the node with the smallest heuristic value.
4. It repeats the process until it reaches the destination or discovers that a route is unavailable.
5. When a blocked path is reported, it can update the map and search for another route.

For example, if Route A appears closer to the destination than Route B, Greedy Best-First Search may choose Route A first. However, Route A could have a storm or a dangerous area that makes it slower or less safe.

Strengths:

- Often finds a route quickly because it is guided by the heuristic.
- Can be useful when an initial answer is needed urgently.
- May explore fewer nodes than an uninformed search in suitable environments.

Weaknesses:

- It does not include the cost already spent when ranking nodes.
- It may choose a short-looking but expensive or dangerous route.
- It does not guarantee the least-cost route.
- Its performance can suffer when the heuristic or environmental information is inaccurate.

1.2 A* Search

A* Search combines the cost already incurred with a heuristic estimate of the remaining cost. Its evaluation function is:

$$f(n) = g(n) + h(n)$$

Here, g(n) is the actual cost from the start to node n, h(n) is the estimated remaining cost, and f(n) is the estimated total cost through n.

The drone calculates the travel cost to candidate locations, estimates the remaining cost, and selects the node with the lowest f(n). Costs can represent travel time, battery use, weather effects, and risk penalties.

Example:

Route	g(n)	h(n)	f(n)
A	8	2	10
B	3	6	9

A* selects Route B because its estimated total cost is 9, compared with 10 for Route A.

Strengths include cost-aware route selection and optimality under suitable conditions. Weaknesses include potentially high memory and computation use, and the need to replan when the environment changes.

1.3 Impact of Heuristic Accuracy

Greedy Best-First Search relies heavily on h(n), so an inaccurate heuristic can direct the drone toward a route that looks close but is unsafe or expensive. A* also uses the heuristic, but combines it with g(n), the actual cost already incurred. An admissible heuristic that does not overestimate the true remaining minimum cost helps A* find an optimal path when the usual search assumptions hold. If weather delays or hazards are omitted from the heuristic and cost model, either algorithm may make inefficient decisions.

1.4 Dynamic Changes and Blocked Paths

Flooding, storms, and restricted airspace may make a previously valid route unusable. Greedy Best-First Search can update the available nodes and choose a new route, but it may again prefer a route that looks close rather than one with the lowest total cost. A* can update edge costs and search for a new least-cost route, although restarting the search after every change can be expensive. Incremental algorithms such as D* Lite can reuse previous search information and are useful when routes change frequently.

1.5 Recommendation

A* with dynamic replanning and explicit safety constraints is recommended. It balances the cost already incurred with the estimated remaining cost. Travel time, battery consumption, and hazard penalties can be included in the cost model. For frequent changes, D* Lite or a similar incremental method can reduce repeated work. Safety restrictions and current sensor information remain essential; a mathematically short route is not acceptable if it is unsafe.

2. Smart City Traffic Signal Optimization

Introduction

A smart city can use AI to adjust signal timings at many intersections to reduce waiting time, fuel consumption, and congestion. Since traffic patterns change continuously and the number of possible signal configurations is large, the task is an optimization problem.

2.1 Formulation as an Optimization Problem

Decision variables can include green-light duration, red-light duration, phase order, and coordination offsets for each intersection. Let x represent the complete set of signal timings.

$$\textbf{Minimize } J(x) = \alpha W(x) + \beta F(x) + \gamma C(x)$$

W(x) is total waiting time, F(x) is fuel consumption, C(x) is a congestion measure, and α, β, and γ are weights representing the importance of each objective. The measures should be normalized or expressed in compatible units.

Constraints include minimum and maximum green times, safe yellow and clearance intervals, non-conflicting signal phases, pedestrian crossing requirements, legal signal rules, and coordination needs between nearby intersections.

Traditional exhaustive search is inefficient because the number of combinations grows rapidly, each configuration requires evaluation, and the best configuration changes as traffic flow changes.

2.2 Hill Climbing

Hill Climbing is a local search method. It starts with a signal configuration, evaluates its traffic cost, creates nearby configurations by changing timings, and moves to a neighboring solution with a lower cost. It repeats until no better neighbor is found or the time limit is reached.

For example, the algorithm may test changing a main-road green light from 40 seconds to 45 seconds. If the overall traffic cost decreases while constraints remain satisfied, it may keep the change.

Advantages: it is simple, uses relatively little memory, and can improve a solution quickly. Limitations: it may become trapped in a local optimum, fail to discover coordinated changes across intersections, and require repeated runs as traffic conditions change.

2.3 Local Optima, Plateaus, and Ridges

Local optimum: a solution better than nearby alternatives but not the best overall. For example, improving one intersection may worsen traffic at nearby intersections, leaving the system with a suboptimal citywide plan.

Plateau: a flat region where nearby timing changes produce almost the same cost. For example, changing a green light by a few seconds may have no measurable effect when traffic is light.

Ridge: a narrow region where improvement requires coordinated changes to several variables. For example, two adjacent intersections may need their green times adjusted together to create a continuous traffic flow. Changing only one may temporarily increase congestion.

For a minimization objective, these are often described as local minima, flat regions, and narrow valleys; the underlying difficulty is the same.

2.4 Simulated Annealing and Genetic Algorithms

Simulated Annealing

Simulated Annealing sometimes accepts a worse candidate solution so that it can escape a local optimum. For a cost increase ΔJ greater than zero, the acceptance probability is $P = e^{(-\Delta J/T)}$, where T is the temperature. At high temperature, worse solutions are more likely to be accepted; as temperature decreases, the algorithm becomes more selective.

It uses limited memory and can explore beyond a local optimum, but its performance depends on the cooling schedule and it may need many evaluations.

Genetic Algorithm

A Genetic Algorithm maintains a population of candidate signal plans. It evaluates them, selects promising candidates, combines them through crossover, and introduces variations through mutation. Repeating these steps can discover useful combinations of timings across intersections.

It explores multiple candidates and can escape some local optima, but computation can be substantial and the algorithm needs careful parameter tuning and constraint handling.

2.5 Recommendation

A hybrid approach is recommended: use a Genetic Algorithm for offline or periodic network-wide optimization, then use a fast local optimizer or traffic controller for real-time refinements. This approach explores many coordinated configurations while keeping urgent signal adjustments fast. All updates must satisfy safety and traffic-law constraints. Simulated Annealing is a reasonable alternative when a single candidate solution and lower memory use are preferred.

3. Autonomous Rover Exploring Mars

Introduction

A Mars rover must explore unknown terrain, collect samples, and avoid hazards such as rocks and craters. It has limited sensing capability and cannot depend on a complete map, so it must make decisions while gathering new information.

3.1 Why Online Search Is Required

Offline search normally calculates a plan from a model assumed to be available before the search begins. An online search agent chooses actions while interacting with the environment, observes new information, updates its model, and replans.

The rover may discover a crater only after approaching an area, or find that a path is too steep to cross. It must then revise its route. Online search is therefore suitable for exploration with incomplete knowledge.

3.2 Partial Observability and Environmental Uncertainty

Partial observability means the rover cannot directly observe the complete state of its surroundings. A camera may miss an obstacle behind a rock, and sensor readings may be noisy. The rover may not know whether terrain is safe until it gets closer.

These conditions can lead to incorrect assumptions, uncertain terrain classification, and risky decisions. The rover should represent unknown regions explicitly, estimate confidence in observations, and avoid actions whose risk exceeds acceptable limits. Its knowledge changes as it senses more of the environment, even when the physical terrain itself changes slowly.

3.3 Building and Updating the Internal Model

1. Sense: collect camera, distance-sensor, and other available measurements.
2. Estimate position and orientation using localization methods.
3. Update a map that records known obstacles, traversable terrain, and unexplored areas.
4. Estimate the cost and risk of possible movements, including energy use.
5. Select a safe and useful next action.
6. Execute the action and gather new observations.
7. Repeat the process and revise the map and plan as needed.

The internal model may include the rover's position, hazards, scientific targets, terrain traversability, energy costs, and confidence in sensor observations. An unknown location can be reclassified after it has been observed.

3.4 Comparison of Online, Uninformed, and Informed Search

Feature	Online Search	Uninformed Search	Informed Search
Knowledge	Learns through interaction	Uses the available model without heuristic guidance	Uses a heuristic to guide search
Planning	Acts and replans while exploring	Often plans on a predefined model	Often plans on a predefined model

Examples	Online DFS, real-time heuristic search	BFS, DFS, UCS	Greedy Best-First, A*
Unknown terrain	Well suited when paired with mapping	Limited if the map is incomplete	Useful when combined with mapping and replanning

These categories describe different properties. Online search concerns when the agent obtains information and chooses actions; informed and uninformed search describe whether heuristic guidance is used. An online agent can also use heuristic-guided planning.

3.5 Recommendation

Use an online search agent with incremental mapping, heuristic path planning, and risk-aware replanning. A* can plan efficient routes through known terrain, while D* Lite can reuse earlier search information when obstacles or path costs change. The rover should also enforce safety limits and monitor battery or energy use. This approach is adaptable to uncertainty, avoids unnecessary repeated computation, and supports safer exploration.

4. University Examination Timetable Using a Constraint Satisfaction Problem

Introduction

A university timetable must assign examination times and halls while preventing student clashes, respecting room capacities, following subject-order requirements, and accommodating faculty availability and special needs. This can be modeled as a Constraint Satisfaction Problem (CSP).

$$\mathbf{CSP = (X, D, C)}$$

X is the set of variables, D is the set of domains, and C is the set of constraints.

4.1 Variables, Domains, and Constraints

Variables

For each examination i, define variables T_i for its time slot and H_i for its hall. For example, Mathematics, Physics, Chemistry, and Computer Science each need a time and an appropriate hall.

Domains

A domain is the set of values a variable can take. For example, T_i may be selected from Monday morning, Monday afternoon, Tuesday morning, and Tuesday afternoon. H_i may be selected from

halls H1, H2, and H3. Unavailable times and unsuitable halls should be removed from the domains.

Constraints

- Student conflicts: if a student takes examinations i and j, then $T_i \neq T$□.
- Hall capacity: the selected hall must have enough seats for the examination.
- Hall availability: one hall cannot host two examinations at the same time.
- Subject ordering: if examination i must occur before j, then $T_i < T$□ in chronological order.
- Faculty availability: an examination requiring a particular faculty member must be scheduled when that person is available.
- Special accommodations: students must be assigned suitable accessible rooms, extra-time arrangements, or other required facilities.
- Duration: the selected time slot must be long enough for the examination.

4.2 Backtracking Search

Backtracking assigns values to variables one at a time. It checks each assignment against the constraints. If a conflict occurs, the algorithm reverses the most recent assignment and tries another value.

1. Select an examination that has not yet been scheduled.
2. Choose a time slot and hall from its current domains.
3. Check whether the assignment satisfies the relevant constraints.
4. If valid, keep the assignment and schedule the next examination.
5. If no valid value remains, return to an earlier examination and change its assignment.
6. Continue until every examination is scheduled or the search determines that no solution exists under the current constraints.

For example, if Mathematics and Physics share students, they cannot occur in the same time slot. If Physics has no other available slot, the algorithm backtracks and changes an earlier assignment.

4.3 Constraint Propagation and Forward Checking

Constraint propagation removes values that cannot be used after another variable is assigned. Forward checking is a common method: after assigning a value, it removes inconsistent values from the domains of connected unassigned variables.

For example, if Mathematics is scheduled on Monday morning, that slot is removed from Physics if students take both subjects. If Physics has no remaining time slots, the algorithm detects the problem immediately and backtracks.

Other useful heuristics are:

- Minimum Remaining Values (MRV): choose the examination with the fewest valid assignments left.
- Degree heuristic: prioritize an examination that constrains the largest number of other examinations.
- Least Constraining Value (LCV): choose the value that removes the fewest options from other variables.

4.4 Real-World Challenges and Scalability

Large universities may have hundreds of examinations, thousands of students, many halls, and changing faculty availability. This creates a large search space. Last-minute changes can also invalidate an otherwise valid timetable.

Hard constraints, such as preventing a student's examinations from overlapping, must always be satisfied. Soft constraints, such as minimizing gaps between a student's examinations or reducing the number of examination days, can be optimized.

$$\textbf{Minimize } J = \alpha G + \beta U + \gamma D$$

Here G is the total student waiting-gap penalty, U is an uneven resource-use penalty, and D is a penalty for using additional examination days. The weights express the university's priorities.

4.5 Recommendation

Use backtracking with forward checking, MRV, the degree heuristic, and LCV. These methods reduce unnecessary search and detect conflicts early. For very large instances, the university may also consider integer programming or specialized timetabling optimization methods. This strategy balances correctness, efficiency, and the ability to revise schedules when circumstances change.

5. Strategic Game AI Using Minimax and Alpha-Beta Pruning

Introduction

A strategy-game AI must choose effective moves while considering possible responses from a human opponent. Since the decision tree can be very large and decisions must be made quickly, Minimax is often combined with Alpha-Beta pruning and time-management techniques.

5.1 Minimax Algorithm

Minimax models a two-player, adversarial game. MAX represents the AI player, which tries to maximize the score, while MIN represents the opponent, which tries to minimize the AI's score.

$$V(n) = U(n), \text{ for a terminal state}$$
$$V(n) = \max V(\text{child}), \text{ at a MAX node}$$
$$V(n) = \min V(\text{child}), \text{ at a MIN node}$$

The algorithm generates possible moves, examines the opponent's responses, evaluates terminal or depth-limited states, and propagates the values back through the tree. The AI selects the move with the highest minimax value.

For example, if Move A has a worst-case value of 2 and Move B has a worst-case value of 5, Minimax selects Move B because it provides the better worst-case outcome.

5.2 Computational Challenges

If b is the average number of moves per state and d is the search depth, ordinary Minimax has approximate time complexity $O(b^d)$. With 10 possible moves at each level and depth 5, a simple estimate is $10^5 = 100{,}000$ nodes.

Real-time strategy games can have many actions involving unit movement, combat, construction, resource collection, and defense. Deep searches are expensive, the game state may change during calculation, and evaluation functions cannot perfectly predict long-term outcomes.

5.3 Alpha-Beta Pruning

Alpha-Beta pruning improves Minimax by skipping branches that cannot affect the final decision. Alpha (α) is the best value MAX can guarantee so far. Beta (β) is the best value MIN can guarantee so far. When $\alpha \geq \beta$, the remaining branches at that point can be pruned.

Detailed example:

1. At a MAX node, evaluate Move A. Its MIN child has outcomes 8 and 5, so MIN chooses 5. MAX's best known value becomes $\alpha = 5$.
2. Next, evaluate Move B. Its MIN child has an outcome of 3, so $\beta = 3$ for that child.
3. Since $\alpha = 5 \geq \beta = 3$, the remaining branches of Move B can be skipped. MIN can already hold Move B to a value no greater than 3, which is worse for MAX than the known value 5 from Move A.
4. The algorithm returns Move A without exploring the unnecessary branches.

Alpha-Beta pruning returns the same minimax decision for the same searched tree and evaluation rules. In the best case, good move ordering can reduce the effective complexity toward O(b^(d/2)); in the worst case, it remains $O(b^d)$.

5.4 Evaluation Function

When the search cannot reach a terminal game state, an evaluation function estimates the position's quality. One possible function is:

$$\mathbf{E(s) = w_1M + w_2R + w_3T + w_4B + w_5P}$$

M is military strength advantage, R is resource advantage, T is territory control, B is base and defensive structure advantage, and P is positional advantage. Each factor should measure the difference between the AI and its opponent, so positive values favor the AI. The weights w_1 through w_5 represent each factor's importance.

- Military strength: number, health, and effectiveness of combat units.
- Resources: ability to build units, develop technology, and sustain the army.
- Territory: control of strategic locations and resource points.
- Bases and defenses: condition of important buildings and defensive structures.
- Position: access to valuable areas, defensive cover, and opportunities to attack.

The weights should be tuned to the game and its stages. The measures should use compatible scales so that one factor does not dominate merely because its numerical values are larger.

5.5 Recommendation for Real-Time Performance

Use iterative deepening Minimax with Alpha-Beta pruning, move ordering, transposition tables, and a strict time limit.

1. Search to depth 1 and save the best move.
2. Increase the search depth gradually.
3. Examine promising moves first to improve pruning.
4. Reuse previous state evaluations with a transposition table.
5. Stop when the time budget is nearly exhausted and return the best move from the deepest completed search.

If a deeper search is interrupted, the AI can still return the move found in the last completed iteration. Quiescence search may also help in unstable positions, such as those involving immediate attacks. For games with simultaneous actions or incomplete information, standard Minimax may need to be replaced or supplemented with a more suitable decision-making method.

This approach preserves Minimax's decision for the searched tree, reduces unnecessary computation, and provides a practical way to meet real-time deadlines.

Overall Conclusion

The five scenarios show that different AI problems require different search and optimization methods. A* with dynamic replanning is suitable for emergency drone routing; a hybrid Genetic Algorithm and fast local optimization can support traffic signal coordination; online search with incremental planning helps a rover explore unknown terrain; CSP with backtracking and

constraint propagation supports valid examination timetables; and iterative deepening Minimax with Alpha-Beta pruning supports strategic game decisions under time limits.

No single algorithm is best for every situation. The choice depends on available information, problem size, time limits, safety requirements, and whether the environment changes. Combining search methods with heuristics, constraints, and real-time updates often produces more practical solutions.